

Secure Secrets, the CLI Way

Student Lab: Azure Key Vault, Blob Storage & a .NET API

All commands use Windows Command Prompt (cmd.exe)

DONE BY: DHARSHINI K

Lab Overview

In this lab you will build a complete, security-first cloud application entirely from the command line. By the end you will have an Azure Key Vault storing a Blob Storage connection string, and an ASP.NET Core Web API that retrieves that secret at startup using a Managed Identity — no passwords, no hard-coded credentials, and no Portal clicking.

What You Will Build

Lab	Goal
Lab 1: CLI Setup	Sign in, set variables, create the resource group
Lab 2: Key Vault	Create an RBAC-enabled vault and test secret read/write
Lab 3: Blob Storage	Create a storage account and store the connection string in Key Vault
Lab 4: .NET API Scaffold	Scaffold an ASP.NET Core Web API and add the Azure packages
Lab 5: Key Vault Integration	Wire Key Vault into the app configuration using DefaultAzureCredential
Lab 6: Blob Endpoints	Build upload and download endpoints that use the Key-Vault-sourced connection string
Lab 7: Deploy with Managed Identity	Deploy the API to App Service and grant it passwordless access to Key Vault
Lab 8: Cleanup	Delete all resources with a single command

Before You Start

- Open a Windows Command Prompt (cmd.exe). Look for plain C:\...> at the prompt — if you see PS C:\...> you are in PowerShell and the variable syntax will be different.
- Confirm the Azure CLI is installed: `az --version`
- Confirm the .NET SDK is installed: `dotnet --version` (8.0 or later required)
- You need an active Azure subscription.

Tip

All variables you set with the `set` command only last for the current session. Keep this `cmd.exe` window open for the whole lab. If you close it and reopen, you will need to re-run the `set` commands from Lab 1.

Important

Key Vault names and Storage account names must be globally unique across all of Azure. Use the `%RANDOM%` trick in Lab 1 to generate a unique suffix — do not change it once you have set it.

Lab 1: CLI Setup

Step 1 — Sign In

1. Run the following command. A browser window will open for you to log in.

```
az login
```

```
dharshinik@DHC47TYX72-dharshinik ~ % az login
A web browser has been opened at https://login.microsoftonline.com/organizations/oauth2/v2.0/authorize.
Please continue the login in the web browser. If no web browser is available or if the web browser fails
to open, use device code flow with `az login --use-device-code`.

Retrieving tenants and subscriptions for the selection...
The following tenants don't contain accessible subscriptions. Use `az login --allow-no-subscriptions` to
have tenant level access.
7f7268a4-7179-4a42-9837-40255b1c21e5 'Presidio Network Solutions'

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Training-2026      e25db867-f63d-489a-8bce-7de21e5f94a1  Presidio Networked Solutions

The default is marked with an *; the default tenant is 'Presidio Networked Solutions' and subscription i
s 'Training-2026' (e25db867-f63d-489a-8bce-7de21e5f94a1).

Select a subscription and tenant (Type a number or Enter for no changes): 1

Tenant: Presidio Networked Solutions
Subscription: Training-2026 (e25db867-f63d-489a-8bce-7de21e5f94a1)
```

Step 2 — Set Your Variables

2. Type each command exactly as shown. Do not add spaces around the = sign.

```
REM Set reusable variables (cmd.exe)
set RG=cli-lab-rg
set LOCATION=eastus
set /A SUFFIX=%RANDOM%
set KV_NAME=cli-lab-kv-%SUFFIX%
set SA_NAME=clilabsa%SUFFIX%
set CONTAINER_NAME=uploads
```

3. Confirm each variable was set correctly:

```
echo %RG%
echo %KV_NAME%
echo %SA_NAME%
```

```

[harshinik@DHC47TYX72-dharshinik ~ % export RG="rg_dharshini"
[harshinik@DHC47TYX72-dharshinik ~ % export location="southindia"
[harshinik@DHC47TYX72-dharshinik ~ % export SUFFIX=$RANDOM
[harshinik@DHC47TYX72-dharshinik ~ % export KV_NAME="dharshini-kv-$SUFFIX"
[harshinik@DHC47TYX72-dharshinik ~ % export SA_NAME="dharshinisa$SUFFIX"
[harshinik@DHC47TYX72-dharshinik ~ % export CONTAINER_NAME="uploads"

[harshinik@DHC47TYX72-dharshinik ~ % echo $RG
rg_dharshini
[harshinik@DHC47TYX72-dharshinik ~ % echo $location
southindia
[harshinik@DHC47TYX72-dharshinik ~ % echo $SUFFIX
1238
[harshinik@DHC47TYX72-dharshinik ~ % echo $suffix

[harshinik@DHC47TYX72-dharshinik ~ % echo $KV_NAME
dharshini-kv-1238
[harshinik@DHC47TYX72-dharshinik ~ % echo $SA_NAME
dharshinisa1238
[harshinik@DHC47TYX72-dharshinik ~ % echo $CONTAINER

[harshinik@DHC47TYX72-dharshinik ~ % echo $CONTAINER_NAME
uploads

```

Write your values here for reference during the rest of the lab:

KV_NAME value	dharshini-kv-1238
----------------------	-------------------

SA_NAME value	dharshinisa1238
----------------------	-----------------

SUFFIX value	1238
---------------------	------

Step 3 — Create the Resource Group

```
az group create --name %RG% --location %LOCATION%
```

```
dharshinik@DHC47TYX72-dharshinik ~ % az group create \  
  --name $RG \  
[  --location $location  
{  
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/rg_dharshini",  
  "location": "southindia",  
  "managedBy": null,  
  "name": "rg_dharshini",  
  "properties": {  
    "provisioningState": "Succeeded"  
  },  
  "tags": null,  
  "type": "Microsoft.Resources/resourceGroups"  
}
```

Lab 2: Create and Secure a Key Vault

Step 1 — Create the Vault

4. Run the following command. The ^ symbol continues the command on the next line — type it exactly as shown.

```
az keyvault create ^  
  --name %KV_NAME% ^  
  --resource-group %RG% ^  
  --location %LOCATION% ^  
  --enable-rbac-authorization true
```

```

dharshinik@DHC47TYX72-dharshinik ~ % az keyvault create \
  --name $KV_NAME \
  --resource-group $RG \
  --location $location \
  [ --enable-rbac-authorization false ]
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/rg_dharshini/providers/Microsoft.KeyVault/vaults/dharshini-kv-1238",
  "location": "southindia",
  "name": "dharshini-kv-1238",
  "properties": {
    "accessPolicies": [
      {
        "applicationId": null,
        "objectId": "12900e06-4032-440a-8e35-46dd3958352a",
        "permissions": {
          "certificates": [
            "all"
          ],
          "keys": [
            "all"
          ],
          "secrets": [
            "all"
          ],
          "storage": [
            "all"
          ]
        },
        "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c"
      }
    ],
    "createMode": null,
    "enablePurgeProtection": null,
    "enableRbacAuthorization": false,
    "enableSoftDelete": true,
    "enabledForDeployment": false,
    "enabledForDiskEncryption": false,
    "enabledForTemplateDeployment": false,
    "hsmPoolResourceId": null,
    "networkAcls": null,
    "privateEndpointConnections": null,
    "provisioningState": "Succeeded",
    "publicNetworkAccess": "Enabled",
    "sku": {
      "family": "A",
      "name": "standard"
    },
    "softDeleteRetentionInDays": 90,
    "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
    "vaultUri": "https://dharshini-kv-1238.vault.azure.net/"
  },
  "resourceGroup": "rg_dharshini",
  "systemData": {
    "createdAt": "2026-07-01T09:35:50.872000+00:00",
    "createdBy": "dkarthik@presidio.com",
    "createdByType": "User",
    "lastModifiedAt": "2026-07-01T09:35:50.872000+00:00",
  }
}

```

Step 2 — Grant Yourself Access

5. Capture your user principal name and subscription ID, then create the role assignment:

```
for /f "delims=" %a in ('az ad signed-in-user show --query userPrincipalName -o tsv') do set MY_UPN=%a
for /f "delims=" %a in ('az account show --query id -o tsv') do set SUBSCRIPTION_ID=%a

az role assignment create ^
  --role "Key Vault Secrets Officer" ^
  --assignee %MY_UPN% ^
  --scope
"/subscriptions/%SUBSCRIPTION_ID%/resourceGroups/%RG%/providers/Microsoft.KeyVault/vaults/%KV_NAME%"
```

Tip

The for /f loop captures command output into a variable. Use single % signs when typing at the prompt (not inside a .bat file).

Important

Role assignments can take 1–2 minutes to propagate. If the next step returns a 403 error, wait a minute and try again.

Skipped this step, as no access

Step 3 — Smoke-Test the Vault

6. Set a test secret and read it back:

```
az keyvault secret set --vault-name %KV_NAME% --name "HelloWorld" --value "it-works"
az keyvault secret show --vault-name %KV_NAME% --name "HelloWorld" --query value -o tsv
```

The second command should print: it-works

7. Delete the test secret:

```
az keyvault secret delete --vault-name %KV_NAME% --name "HelloWorld"
```

```

dharshinik@DHC47TYX72-dharshinik ~ % az keyvault secret set \
  --vault-name $KV_NAME \
  --name HelloWorld \
  --value it-works

az keyvault secret show \
  --vault-name $KV_NAME \
  --name HelloWorld \
  --query value \
  -o tsv
[
  {
    "attributes": {
      "created": "2026-07-01T09:38:12+00:00",
      "enabled": true,
      "expires": null,
      "notBefore": null,
      "recoverableDays": 90,
      "recoveryLevel": "Recoverable+Purgeable",
      "updated": "2026-07-01T09:38:12+00:00"
    },
    "contentType": null,
    "id": "https://dharshini-kv-1238.vault.azure.net/secrets/HelloWorld/56032a6eb4ab4cf6bd42a5164d42ddc8",
    "kid": null,
    "managed": null,
    "name": "HelloWorld",
    "tags": {
      "file-encoding": "utf-8"
    },
    "value": "it-works"
  }
]
it-works
dharshinik@DHC47TYX72-dharshinik ~ % az keyvault secret show \
  --vault-name $KV_NAME \
  --name HelloWorld \
  --query value \
  -o tsv
[
  it-works
]
dharshinik@DHC47TYX72-dharshinik ~ %

```

```

dharshinik@DHC47TYX72-dharshinik ~ % az keyvault secret list \
  --vault-name $KV_NAME \
  --output table
Name                               Id                               ContentType  Enabled  Expires
-----
HelloWorld https://dharshini-kv-1238.vault.azure.net/secrets/HelloWorld True
dharshinik@DHC47TYX72-dharshinik ~ % az keyvault secret delete \
  --vault-name $KV_NAME \
  --name HelloWorld
Warning! If you have soft-delete protection enabled on this key vault, this secret will be moved to the soft deleted state. Your secret has been purged from the soft-deleted state. Please see the following documentation for additional guidance.
https://learn.microsoft.com/azure/key-vault/general/soft-delete-overview
{
  "attributes": {
    "created": "2026-07-01T09:38:12+00:00",
    "enabled": true,
    "expires": null,
    "notBefore": null,
    "recoverableDays": 90,
    "recoveryLevel": "Recoverable+Purgeable",
    "updated": "2026-07-01T09:38:12+00:00"
  },
  "contentType": null,
  "deletedDate": "2026-07-01T09:43:19+00:00",
  "id": "https://dharshini-kv-1238.vault.azure.net/secrets/HelloWorld/56032a6eb4ab4cf6bd42a5164d42ddc8",
  "kid": null,
  "managed": null,
  "name": "HelloWorld",
  "recoveryId": "https://dharshini-kv-1238.vault.azure.net/deletedsecrets/HelloWorld",
  "scheduledPurgeDate": "2026-09-29T09:43:19+00:00",
  "tags": {
    "file-encoding": "utf-8"
  },
  "value": null
}
dharshinik@DHC47TYX72-dharshinik ~ % az keyvault secret list \
  --vault-name $KV_NAME \
  --output table
dharshinik@DHC47TYX72-dharshinik ~ %

```

Command 1: storing a secret with name “Hello World” and value “it works” (key : value pairs)

Command 2: accessing value by name

Command 3: deleting the secret pair

Lab 3: Blob Storage and the Connection String Secret

Step 1 — Create the Storage Account and Container

```
az storage account create ^  
  --name %SA_NAME% ^  
  --resource-group %RG% ^  
  --location %LOCATION% ^  
  --sku Standard_LRS ^  
  --kind StorageV2  
  
az storage container create ^  
  --name %CONTAINER_NAME% ^  
  --account-name %SA_NAME% ^  
  --auth-mode login
```

```

dharshinik@DHC47TYX72-dharshinik ~ % az storage account create \
  --name $SA_NAME \
  --resource-group $RG \
  --location $location \
  --sku Standard_LRS \
  --kind StorageV2
[
  {
    "accessTier": "Hot",
    "accountMigrationInProgress": null,
    "allowBlobPublicAccess": false,
    "allowCrossTenantReplication": false,
    "allowSharedKeyAccess": null,
    "allowSharedKeyAccessForServices": null,
    "allowedCopyScope": null,
    "azureFilesIdentityBasedAuthentication": null,
    "blobRestoreStatus": null,
    "creationTime": "2026-07-01T09:45:51.693030+00:00",
    "customDomain": null,
    "dataCollaborationPolicyProperties": null,
    "defaultToOAuthAuthentication": null,
    "dnsEndpointType": null,
    "dualStackEndpointPreference": null,
    "enableExtendedGroups": null,
    "enableHttpsTrafficOnly": true,
    "enableNfsV3": null,
    "encryption": {
      "encryptionIdentity": null,
      "keySource": "Microsoft.Storage",
      "keyVaultProperties": null,
      "requireInfrastructureEncryption": null,
      "services": {
        "blob": {
          "enabled": true,
          "keyType": "Account",
          "lastEnabledTime": "2026-07-01T09:45:51.795465+00:00"
        }
      }
    },
  }
],

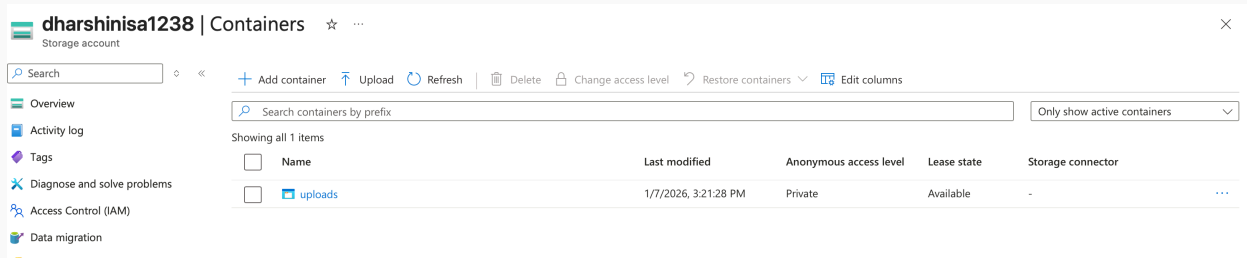
```

Command 1: Create a Storage Account with sku (**stock keeping unit = Standard_LRS** => Standard Performance, Locally Redundant Storage (replicas created in same data center) and **kind = StorageV2** (General Purpose)

```

dharshinik@DHC47TYX72-dharshinik ~ % az storage container create \
  --name $CONTAINER_NAME \
  --account-name $SA_NAME \
  --auth-mode login
[
  {
    "created": true
  }
]

```



Command 2: Create a container with the storage account

Step 2 — Capture the Connection String

```
for /f "delims=" %a in ('az storage account show-connection-string --name %SA_NAME% --resource-group %RG% --query connectionString -o tsv') do set CONN_STRING=%a
```

Tip

Verify the variable was captured: `echo %CONN_STRING%` — it should start with `DefaultEndpointsProtocol=https`

Step 3 — Store the Connection String in Key Vault

```
az keyvault secret set ^
--vault-name %KV_NAME% ^
--name "BlobStorageConnectionString" ^
--value "%CONN_STRING%"
```

```

dharshinik@DHC47TYX72-dharshinik ~ % export CONN_STRING=$(az storage account show-connection-string \
--name $SA_NAME \
--resource-group $RG \
--query connectionString \
-o tsv)
dharshinik@DHC47TYX72-dharshinik ~ % echo $CONN_STRING
DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=dharshinisa1238;AccountKey=71
AtlgAfqCqEQ466vibK53Hr0DyJNilaENRpiLd54KgoPL4vt1fesmRm+S6xofSDSJy88fLHGGU6+AStQsaKMw==;BlobEndpoint=http
s://dharshinisa1238.blob.core.windows.net/;FileEndpoint=https://dharshinisa1238.file.core.windows.net/;Q
ueueEndpoint=https://dharshinisa1238.queue.core.windows.net/;TableEndpoint=https://dharshinisa1238.table
.core.windows.net/
dharshinik@DHC47TYX72-dharshinik ~ %

```

This is the connection string of Storage Account (generated on creation)

```

[harshinik@DHC47TYX72-dharshinik BlobVaultApi % az keyvault secret list \
  --vault-name $KV_NAME \
  -o table
]

[harshinik@DHC47TYX72-dharshinik BlobVaultApi % echo $CONN_STRING
DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=dharshinisa1238;AccountKey=71
AtlgAfqCqEQ466vibK53Hr0DyJNilaENRpiLd54KgoPL4vt1fesmRm+S6xofSDSJy88fLHGGU6+ASTQsaKMw==;BlobEndpoint=http
s://dharshinisa1238.blob.core.windows.net/;FileEndpoint=https://dharshinisa1238.file.core.windows.net/;Q
ueueEndpoint=https://dharshinisa1238.queue.core.windows.net/;TableEndpoint=https://dharshinisa1238.table
.core.windows.net/
dharshinik@DHC47TYX72-dharshinik BlobVaultApi % az keyvault secret set \
  --vault-name $KV_NAME \
  --name BlobStorageConnectionString \
  --value "$CONN_STRING"
]
{
  "attributes": {
    "created": "2026-07-01T10:47:05+00:00",
    "enabled": true,
    "expires": null,
    "notBefore": null,
    "recoverableDays": 90,
    "recoveryLevel": "Recoverable+Purgeable",
    "updated": "2026-07-01T10:47:05+00:00"
  },
  "contentType": null,
  "id": "https://dharshini-kv-1238.vault.azure.net/secrets/BlobStorageConnectionString/e6e3d4667b0b4b8a9
e72e1cb2aba32bd",
  "kid": null,
  "managed": null,
  "name": "BlobStorageConnectionString",
  "tags": {
    "file-encoding": "utf-8"
  },
  "value": "DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=dharshinisa1238;A
ccountKey=71AtlgAfqCqEQ466vibK53Hr0DyJNilaENRpiLd54KgoPL4vt1fesmRm+S6xofSDSJy88fLHGGU6+ASTQsaKMw==;Blobe
ndpoint=https://dharshinisa1238.blob.core.windows.net/;FileEndpoint=https://dharshinisa1238.file.core.wi
ndows.net/;QueueEndpoint=https://dharshinisa1238.queue.core.windows.net/;TableEndpoint=https://dharshini
sa1238.table.core.windows.net/"
}

```

Set connection String in Key Vault

Important

This is the last time the connection string will be handled directly. From here on, only Key Vault knows it.

Lab 4: Scaffold the .NET Web API

Step 1 — Create the Project

Tip

Choose a folder on your machine where you want to create the project, then cd into it before running these commands.

```
dotnet new webapi -n BlobVaultApi --use-controllers
cd BlobVaultApi
```

Step 2 — Add the Azure NuGet Packages

```
dotnet add package Azure.Identity
dotnet add package Azure.Extensions.AspNetCore.Configuration.Secrets
dotnet add package Azure.Storage.Blobs
```

```
[dharshinik@DHC47TYX72-dharshinik SecureSecrets % dotnet new webapi -n BlobVaultApi --use-controllers ]
zsh: command not found: dotnet
[dharshinik@DHC47TYX72-dharshinik SecureSecrets % dotnet new webapi -n BlobVaultApi --use-controllers ]
The template "ASP.NET Core Web API" was created successfully.

Processing post-creation actions...
Restoring /Users/dharshinik/Desktop/Presidio/Genspark/Azure/SecureSecrets/BlobVaultApi/BlobVaultApi.csproj
obj:
/Users/dharshinik/Desktop/Presidio/Genspark/Azure/SecureSecrets/BlobVaultApi/BlobVaultApi.csproj : warning
NU1903: Package 'Microsoft.OpenApi' 2.0.0 has a known high severity vulnerability, https://github.com
/advisor/GHSA-v5pm-xwqc-g5wc
Restore succeeded.

[dharshinik@DHC47TYX72-dharshinik SecureSecrets % cd BlobVaultApi ]
dharshinik@DHC47TYX72-dharshinik BlobVaultApi % dotnet add package Azure.Identity
dotnet add package Azure.Extensions.AspNetCore.Configuration.Secrets
dotnet add package Azure.Storage.Blobs

info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/
dotnet/sdk/10.0.202/trustedroots/codesignctl.pem'.
info : X.509 certificate chain validation will use the fallback certificate bundle at '/usr/local/share/
dotnet/sdk/10.0.202/trustedroots/timestampctl.pem'.
info : Adding PackageReference for package 'Azure.Identity' into project '/Users/dharshinik/Desktop/Pres
idio/Genspark/Azure/SecureSecrets/BlobVaultApi/BlobVaultApi.csproj'.
info : GET https://api.nuget.org/v3/registration5-gz-semver2/azure.identity/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/azure.identity/index.json 472ms
info : Restoring packages for /Users/dharshinik/Desktop/Presidio/Genspark/Azure/SecureSecrets/BlobVaultA
pi/BlobVaultApi.csproj...
```

Package

Purpose

Azure.Identity	DefaultAzureCredential — uses your az login locally, Managed Identity in Azure
Azure.Extensions.AspNetCore.Configuration.Secrets	Maps Key Vault secrets directly into ASP.NET Core configuration
Azure.Storage.Blobs	Blob Storage client for upload and download

Lab 5: Wire Up Key Vault in Program.cs

Step 1 — Edit Program.cs

8. Open BlobVaultApi/Program.cs in a text editor or IDE.
9. Replace the entire file content with the following:

```
using Azure.Identity;

var builder = WebApplication.CreateBuilder(args);

var keyVaultUri = new
Uri($"https://{builder.Configuration["KeyVaultName"]}.vault.azure.net/");
builder.Configuration.AddAzureKeyVault(keyVaultUri, new DefaultAzureCredential());

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Step 2 — Set the Vault Name in App Settings

10. Open BlobVaultApi/appsettings.Development.json and replace the KeyVaultName value with YOUR vault name (the value you wrote down from Lab 1):

```
{
  "KeyVaultName": "YOUR-KV-NAME-HERE",
  "Logging": { "LogLevel": { "Default": "Information" } }
}
```

Important

Replace YOUR-KV-NAME-HERE with the actual value from your KV_NAME fill-in box on page 2. Using the wrong name will cause a connection error when you run the app.

Step 3 — Verify the App Starts

```
dotnet run
```

Watch the startup log. You should see no errors. The app will print a localhost URL — leave it running.

```
dhharshinik@DHC47TYX72-dharshinik BlobVaultApi % dotnet run startup
Using launch settings from /Users/dharshinik/Desktop/Presidio/Genspark/Azure/SecureSecrets/BlobVaultApi/Properties/launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5179
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Users/dharshinik/Desktop/Presidio/Genspark/Azure/SecureSecrets/BlobVaultApi
```

Lab 6: Blob Upload and Download Endpoints

Step 1 — Register the Blob Service Client

11. Add the following to Program.cs, after the AddAzureKeyVault line and before builder.Build():

```
using Azure.Storage.Blobs;

builder.Services.AddSingleton(x =>
{
    var connStr = builder.Configuration["BlobStorageConnectionString"];
    return new BlobServiceClient(connStr);
});
```

Step 2 — Create the Files Controller

12. Create a new file: BlobVaultApi/Controllers/FilesController.cs
13. Paste in the following code:

```
using Microsoft.AspNetCore.Mvc;
using Azure.Storage.Blobs;

namespace BlobVaultApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class FilesController : ControllerBase
{
    private readonly BlobContainerClient _container;
    private const string ContainerName = "uploads";

    public FilesController(BlobServiceClient blobServiceClient)
    {
        _container = blobServiceClient.GetBlobContainerClient(ContainerName);
        _container.CreateIfNotExists();
    }

    [HttpPost("upload")]
    public async Task<IActionResult> Upload(IFormFile file)
    {
        if (file is null || file.Length == 0)
            return BadRequest("No file provided.");

        var blobClient = _container.GetBlobClient(file.FileName);
        using var stream = file.OpenReadStream();
        await blobClient.UploadAsync(stream, overwrite: true);

        return Ok(new { fileName = file.FileName, url = blobClient.Uri.ToString() });
    }
}
```

```
[HttpGet("download/{fileName}")]
public async Task<IActionResult> Download(string fileName)
{
    var blobClient = _container.GetBlobClient(fileName);
    if (!await blobClient.ExistsAsync())
        return NotFound();

    var download = await blobClient.DownloadContentAsync();
    return File(download.Value.Content.ToArray(), "application/octet-stream",
fileName);
}
}
```

Step 3 — Run and Test

14. Start the app:

```
dotnet run
```

15. Open a SECOND cmd.exe window (leave dotnet run running in the first). Test the upload endpoint — replace sample.jpg with any image or file you have:

```
curl -F "file=@sample.jpg" https://localhost:5001/api/files/upload -k
```

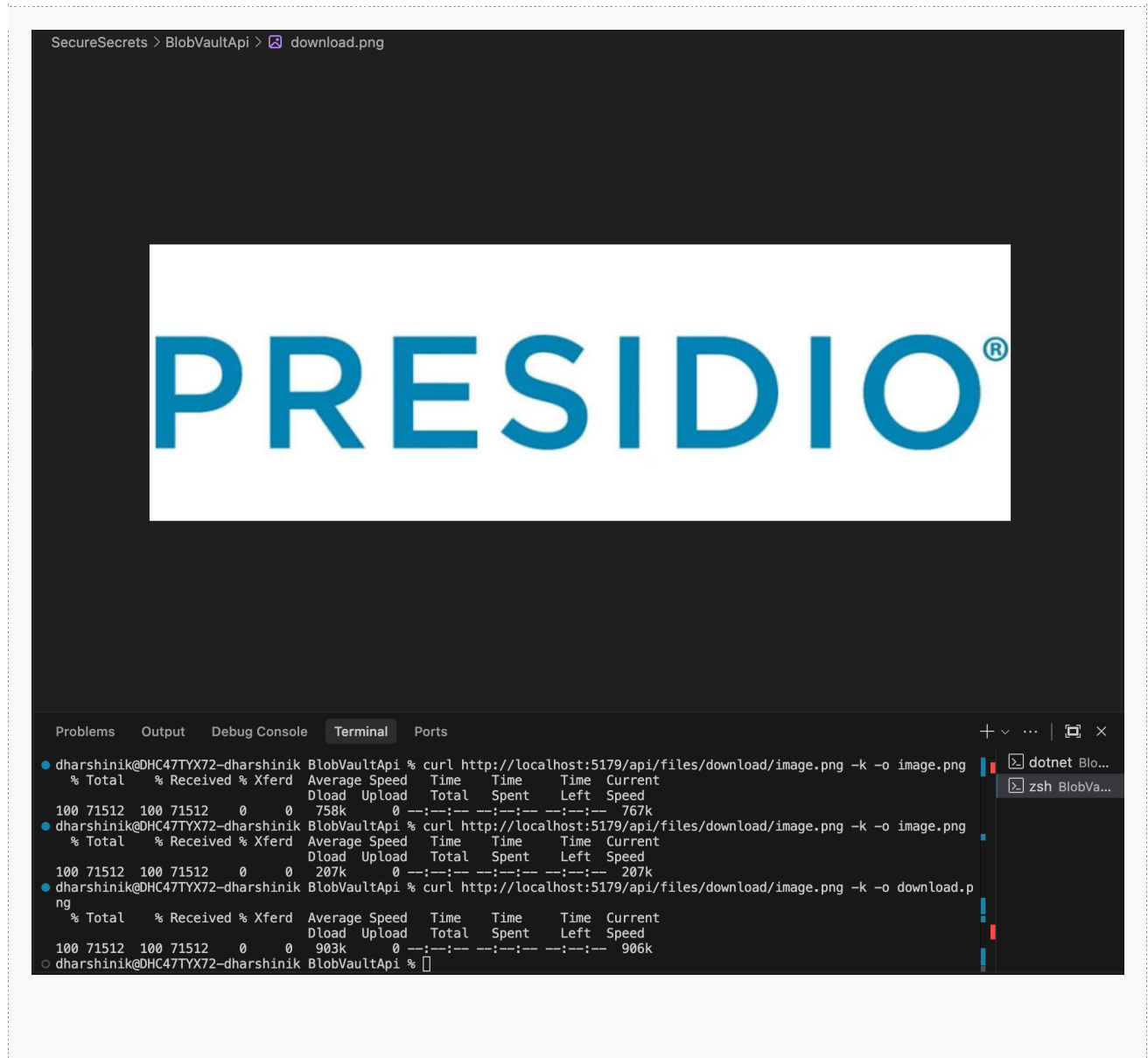
16. Test the download endpoint:

```
curl https://localhost:5001/api/files/download/sample.jpg -k -o downloaded.jpg
```

```
dharshinik@DHC47TYX72-dharshinik BlobVaultApi % curl -F "file=@image.png" http://localhost:5179/api/files/upload
{"fileName":"image.png","url":"https://dharshinisa1238.blob.core.windows.net/uploads/image.png"}%
dharshinik@DHC47TYX72-dharshinik BlobVaultApi %
```

The screenshot shows the Azure Storage Explorer interface for a container named 'uploads'. The left sidebar has 'Overview' selected. The main pane shows the container's details, including the authentication method (Access key) and a search bar. Below, a table lists the blobs in the container.

Name	Last modified	Access tier	Blob type	Size	Lease state
image.png	1/7/2026, 4:18:18 PM	Hot (Inferred)	Block blob	69.84 KiB	Available



Lab 7: Deploy with a Managed Identity

Step 1 — Create the App Service Plan and Web App

```
set APP_NAME=cli-lab-api-%SUFFIX%

az appservice plan create ^
  --name cli-lab-plan ^
  --resource-group %RG% ^
  --sku B1 ^
  --is-linux

az webapp create ^
  --name %APP_NAME% ^
  --resource-group %RG% ^
  --plan cli-lab-plan ^
  --runtime "DOTNETCORE:8.0"
```

```
dharshinik@DHC47TYX72-dharshinik ~ % az webapp create \
  --name "$APP_NAME" \
  --resource-group "$RG" \
  --plan dharshini-plan \
  --runtime "DOTNETCORE:10.0"
Webapp 'dharshini-api-1238' created. Deploy your code with: az webapp deploy
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": true,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 0,
  "customDomainVerificationId": "46807E15E4B3288A1BCC91EA4477FF9CF6E3CE676FACDEA0663A4E096E2CCC01",
  "dailyMemoryTimeQuota": 0,
  "daprConfig": null,
  "defaultHostName": "dharshini-api-1238.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  },
}
```

Creates an App Service Plan.

Think of an App Service Plan as the VM infrastructure that hosts your web apps.

Creates a .NET 8 web application hosted on my App Service Plan.

APP_NAME value	dharshini-api-1238
-----------------------	--------------------

Step 2 — Configure the App Setting

```
az webapp config appsettings set ^
  --name %APP_NAME% ^
  --resource-group %RG% ^
  --settings KeyVaultName=%KV_NAME%
```

Step 3 — Assign a Managed Identity

```
az webapp identity assign --name %APP_NAME% --resource-group %RG%
```

17. Capture the identity's principal ID:

```
for /f "delims=" %a in ('az webapp identity show --name %APP_NAME% --resource-group %RG% -
-query principalId -o tsv') do set PRINCIPAL_ID=%a
```

```

dharshinik@DHC47TYX72-dharshinik ~ % az webapp identity assign \
  --name "$APP_NAME" \
  --resource-group "$RG"
{
  "principalId": "39ce0d08-d73d-44ee-b579-6bb255f48c69",
  "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
  "type": "SystemAssigned",
  "userAssignedIdentities": null
}
dharshinik@DHC47TYX72-dharshinik ~ % export PRINCIPAL_ID=$(az webapp identity show \
  --name "$APP_NAME" \
  --resource-group "$RG" \
  --query principalId \
  -o tsv)

echo $PRINCIPAL_ID
39ce0d08-d73d-44ee-b579-6bb255f48c69

```

Step 4 — Grant the Identity Key Vault Access

```
az role assignment create ^
  --role "Key Vault Secrets User" ^
  --assignee %PRINCIPAL_ID% ^
  --scope
"/subscriptions/%SUBSCRIPTION_ID%/resourceGroups/%RG%/providers/Microsoft.KeyVault/vaults/%KV_NAME%"
```

Step 5 — Publish and Deploy

cmd.exe has no built-in zip command, so you will call PowerShell just for the compression step:

```
dotnet publish -c Release -o .\publish
cd publish
powershell -Command "Compress-Archive -Path * -DestinationPath ..\publish.zip -Force"
cd ..

az webapp deploy ^
  --name %APP_NAME% ^
  --resource-group %RG% ^
  --src-path publish.zip ^
  --type zip
```

```
dharshinik@DHC47TYX72-dharshinik BlobVaultApi % dotnet publish -c Release -o .\publish
Restore complete (0.4s)
  BlobVaultApi net10.0 succeeded (3.2s) → publish/

Build succeeded in 4.2s
dharshinik@DHC47TYX72-dharshinik BlobVaultApi %
```

copy the files from local to the web app for deployment

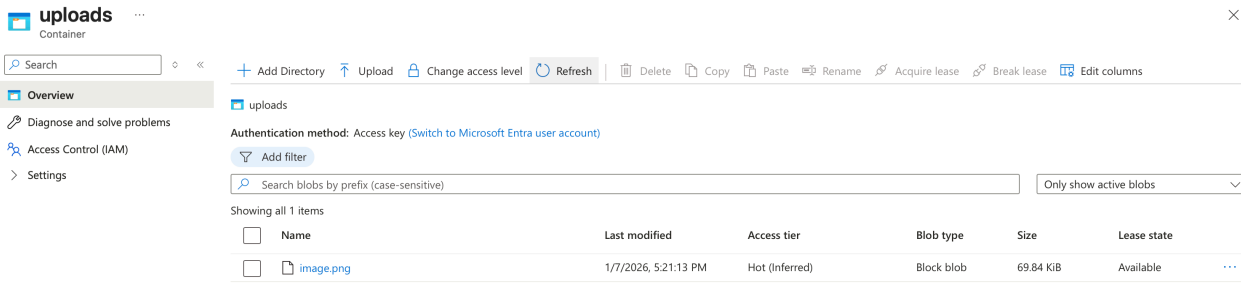
```

dharshinik@DHC47TYX72-dharshinik BlobVaultApi % cd publish
dharshinik@DHC47TYX72-dharshinik publish % zip -r ../publish.zip .
  adding: Azure.Storage.Common.dll (deflated 52%)
  adding: System.ClientModel.dll (deflated 57%)
  adding: Azure.Extensions.AspNetCore.Configuration.Secrets.dll (deflated 45%)
  adding: Azure.Core.dll (deflated 64%)
  adding: Azure.Identity.dll (deflated 45%)
  adding: web.config (deflated 39%)
  adding: runtimes/ (stored 0%)
  adding: runtimes/win/ (stored 0%)
  adding: runtimes/win/lib/ (stored 0%)
  adding: runtimes/win/lib/netstandard2.0/System.Security.Cryptography.ProtectedData.dll (deflated 45%)
  adding: Microsoft.OpenApi.dll (deflated 62%)
  adding: appsettings.json (deflated 26%)
  adding: BlobVaultApi.deps.json (deflated 78%)
  adding: Azure.Storage.Blobs.dll (deflated 67%)
  adding: System.IO.Hashing.dll (deflated 47%)
  adding: BlobVaultApi.staticwebassets.endpoints.json (stored 0%)
  adding: Microsoft.AspNetCore.OpenApi.dll (deflated 57%)
  adding: BlobVaultApi.runtimeconfig.json (deflated 50%)
  adding: Swashbuckle.AspNetCore.SwaggerUI.dll (deflated 9%)
  adding: Swashbuckle.AspNetCore.SwaggerGen.dll (deflated 58%)
  adding: Microsoft.IdentityModel.Abstractions.dll (deflated 43%)
  adding: Microsoft.Identity.Client.dll (deflated 67%)
  adding: System.Memory.Data.dll (deflated 46%)
  adding: BlobVaultApi.dll (deflated 56%)
  adding: System.Security.Cryptography.ProtectedData.dll (deflated 44%)
  adding: appsettings.Development.json (deflated 25%)
  adding: BlobVaultApi (deflated 75%)
  adding: Microsoft.Identity.Client.Extensions.Msal.dll (deflated 55%)
  adding: Azure.Security.KeyVault.Secrets.dll (deflated 57%)
  adding: Swashbuckle.AspNetCore.Swagger.dll (deflated 56%)
  adding: Microsoft.Bcl.AsyncInterfaces.dll (deflated 42%)
  adding: BlobVaultApi.pdb (deflated 40%)
dharshinik@DHC47TYX72-dharshinik publish %

```

Step 6 — Test in the Cloud

```
curl -F "file=@sample.jpg" https://%APP_NAME%.azurewebsites.net/api/files/upload
```



The screenshot shows the Azure Storage Explorer interface for a container named 'uploads'. The 'Overview' tab is selected, displaying a table of blobs. The table has columns for Name, Last modified, Access tier, Blob type, Size, and Lease state. One item, 'image.png', is listed with a size of 69.84 KiB and a lease state of 'Available'.

Name	Last modified	Access tier	Blob type	Size	Lease state
image.png	1/7/2026, 5:21:13 PM	Hot (Inferred)	Block blob	69.84 KiB	Available

```

dharshinik@DHC47TYX72-dharshinik BlobVaultApi % curl -F "file=@image.png" https://$APP_NAME.azurewebsites.net/api/files/upload
{"fileName":"image.png","url":"https://dharshinisa1238.blob.core.windows.net/uploads/image.png"}
dharshinik@DHC47TYX72-dharshinik BlobVaultApi %

```



Lab 8: Cleanup

Important

This step permanently deletes all resources created today. Only run it when you are completely finished with the lab.

```
az group delete --name %RG% --yes --no-wait
```

```
dharshinik@DHC47TYX72-dharshinik BlobVaultApi % az group exists --name rg_dharshini
false
dharshinik@DHC47TYX72-dharshinik BlobVaultApi %
```

Reflection Questions

Answer the following questions to check your understanding of the lab.

1. Why is the Blob Storage connection string stored in Key Vault instead of in appsettings.json?

The Blob Storage connection string is stored in Azure Key Vault to keep sensitive information separate from the application code and configuration files. This improves security because secrets are centrally managed, encrypted, and access-controlled. If the connection string changes, it can be updated in Key Vault without modifying or redeploying the application.

2. What is DefaultAzureCredential, and why does the same line of code work both locally and in Azure?

It is an authentication class from the Azure Identity library that automatically selects the appropriate authentication method based on the environment. When running locally, it uses the Azure CLI or Visual Studio login. When deployed to Azure App Service, it automatically uses the application's Managed Identity. Because it detects the available credential source automatically, the same code works in both environments without modification.

3. Why does the App Service identity get the Key Vault Secrets User role instead of Secrets Officer?

to retrieve the Blob Storage connection string. The Secrets Officer role allows creating, updating, and deleting secrets, which violates the principle of least privilege. Granting only the minimum required permissions reduces the risk of accidental or malicious changes.

4. What would happen if you accidentally committed appsettings.json with the raw connection string to a public GitHub repository? How does this lab's approach prevent that?

If the raw connection string were committed to a public GitHub repository, anyone could use it to access the Azure Storage Account, potentially reading, modifying, or deleting data and increasing Azure costs. In this lab, appsettings.json stores only the Key Vault name, while the

actual connection string is stored securely in Azure Key Vault. The application retrieves the secret at runtime using Managed Identity, so no sensitive credentials are stored in the source code or Git repository.

Common Errors

Error	Fix
InvalidResourceGroup: name has invalid character '\$'	You are using PowerShell/bash variable syntax. Switch to cmd.exe and use %VAR% syntax
No such host known (vault.azure.net)	The vault name in appsettings.Development.json is still the placeholder. Replace it with your actual KV_NAME value
403 Forbidden (Key Vault)	Role assignment hasn't propagated yet — wait 60 seconds and retry
DefaultAzureCredential fails	Run az login again in the same cmd.exe window
curl upload returns 401/403	The App Service Managed Identity role assignment may not have propagated — wait 2 minutes